# TRAVAUX PRATIQUES GUIDÉS

Terraform sur AWS — de l'Infrastructure as Code aux modules

Parcours progressif : IaC • Terraform • Provider • Ressources • Outputs • Variables • Interpolation • Modules

Public : stagiaires ayant découvert les bases de Terraform

Durée indicative : 3 h 30 à 4 h 30, selon l'aisance du groupe et les échanges

Plateforme : Amazon Web Services (AWS)

Version : la version exacte de Terraform n'ayant pas été communiquée, le TP utilise une contrainte Terraform 1.x (>= 1.5.0, < 2.0.0) et un provider AWS 6.x. Vérifier les versions réellement installées avant la séance.

Scénario : construire une petite infrastructure web de démonstration dans AWS : un VPC, un sous-réseau public, une passerelle Internet, une table de routage, un groupe de sécurité et une instance EC2. Puis rendre le projet paramétrable et extraire l'infrastructure dans un module réutilisable.

Document stagiaire — les étapes sont guidées, mais les questions de contrôle doivent être traitées avant de passer à la suivante.

# 1. Objectifs pédagogiques

• Expliquer l'intérêt de l'Infrastructure as Code et de la démarche déclarative.

• Initialiser un projet Terraform et comprendre le rôle du provider AWS.

• Déclarer des ressources AWS et comprendre leurs dépendances.

• Lire un plan, appliquer une configuration et détruire proprement les ressources.

• Exposer des attributs avec les outputs.

• Paramétrer le déploiement avec des variables, des valeurs locales et des fichiers tfvars.

• Utiliser l'interpolation et les références entre ressources.

• Factoriser la configuration dans un module local réutilisable.

# 2. Architecture cible

L'infrastructure finale est volontairement simple. Elle ne déploie pas de serveur web applicatif : l'instance EC2 sert à illustrer le provisionnement des ressources et les liens entre elles.

• Un VPC avec un bloc CIDR configurable.

• Un sous-réseau public dans une zone de disponibilité.

• Une Internet Gateway attachée au VPC.

• Une table de routage avec une route par défaut vers Internet Gateway.

• Une association entre la table de routage et le sous-réseau.

• Un Security Group autorisant SSH depuis une adresse IP explicitement autorisée et HTTP depuis Internet.

• Une instance EC2 utilisant une AMI Amazon Linux 2023 recherchée dynamiquement.

Coût et sécurité : des ressources AWS réelles peuvent engendrer des frais. L'instance EC2 est de type t3.micro par défaut, mais sa facturation dépend de la région, de l'éligibilité Free Tier et du compte. Le TP limite SSH à votre IP publique ; ne remplacez pas cette restriction par 0.0.0.0/0.

# 3. Pré-requis

• Un compte AWS autorisé à créer VPC, sous-réseaux, route tables, Internet Gateways, Security Groups et instances EC2.

• AWS CLI installé et configuré, ou des identifiants fournis par le formateur via un mécanisme sécurisé.

• Terraform installé (la commande terraform version doit fonctionner).

• Un terminal et un éditeur de texte.

• Une adresse IPv4 publique fixe ou connue pour autoriser SSH ; le TP demandera de la saisir au format CIDR, par exemple 203.0.113.10/32 (exemple documentaire, à remplacer).

# 3.1 Vérifier les outils

```
terraform version
aws --version
aws sts get-caller-identity
```

La dernière commande doit afficher l'identité AWS utilisée. Si elle échoue, corrigez l'authentification avant de poursuivre.

# 3.2 Authentification AWS

Utilisez de préférence un profil AWS CLI déjà configuré ou un rôle IAM temporaire. Ne placez jamais de clé d’accès ou de secret dans les fichiers .tf, dans un dépôt Git ou dans le PDF. Terraform utilise la chaîne standard de résolution des identifiants AWS.

```
aws configure --profile formation
aws sts get-caller-identity --profile formation
# Puis, dans le terminal de travail :
export AWS_PROFILE=formation
```

Sous PowerShell, utilisez : $env:AWS_PROFILE="formation". Si votre environnement utilise déjà un rôle IAM, ne lancez pas aws configure.

# 4. Étape 1 — Créer le projet et initialiser Terraform

Objectif : créer le répertoire de travail, déclarer le provider AWS et initialiser Terraform.

```
mkdir tp-terraform-aws
cd tp-terraform-aws
```

## 4.1 Créer le fichier versions.tf

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}
```

## 4.2 Créer le fichier provider.tf

```
provider "aws" {
  region = var.aws_region
}
```

## 4.3 Créer le fichier variables.tf

```
variable "aws_region" {
  description = "Région AWS de déploiement"
  type        = string
  default     = "eu-west-3"
}
```

## 4.4 Initialiser

```
terraform init
terraform version
terraform providers
```

À observer : Terraform télécharge le provider déclaré et crée le répertoire .terraform ainsi que le fichier .terraform.lock.hcl. Le fichier de verrouillage enregistre les versions sélectionnées ; il doit être conservé dans le dépôt.

Questions : Quel est le rôle de terraform init ? Quelle différence entre Terraform Core et le provider AWS ? Pourquoi versionner .terraform.lock.hcl ?

# 5. Étape 2 — Déclarer les premières ressources AWS

Objectif : construire le réseau nécessaire à l'instance EC2. Les fichiers .tf placés à la racine constituent ensemble le module racine.

## 5.1 Ajouter les variables réseau dans variables.tf

```
variable "vpc_cidr" {
  description = "Plage CIDR du VPC"
  type        = string
  default     = "10.20.0.0/16"
}

variable "public_subnet_cidr" {
  description = "Plage CIDR du sous-réseau public"
  type        = string
  default     = "10.20.1.0/24"
```

```
}

variable "availability_zone" {
  description = "Zone de disponibilité AWS"
  type        = string
  default     = "eu-west-3a"
}

variable "ssh_allowed_cidr" {
  description = "Adresse IPv4 autorisée pour SSH au format CIDR /32"
  type        = string
  sensitive   = false

  validation {
    condition     = can(cidrnetmask(var.ssh_allowed_cidr)) && can(regex("/32$", var.ssh_allowed_cidr))
    error_message = "Fournissez une adresse IPv4 au format x.x.x.x/32."
  }
}
```

## 5.2 Créer network.tf

```
resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name        = "tp-terraform-vpc"
    Environment = "formation"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "tp-terraform-public-subnet"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id

  tags = {
    Name = "tp-terraform-igw"
  }
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }

  tags = {
    Name = "tp-terraform-public-rt"
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}
```

Remarquez les références comme aws_vpc.main.id. Elles expriment une dépendance : le sous-réseau utilise l'identifiant du VPC. Terraform construit ainsi son graphe de dépendances et détermine l'ordre des opérations.

## 5.3 Valider et examiner le plan

```
terraform fmt
terraform validate
```

```
terraform plan
```

À ce stade, le plan doit annoncer la création des ressources réseau. Il n’y a encore aucune modification dans AWS : plan ne fait qu’afficher les opérations envisagées.

Questions : Pourquoi le sous-réseau référence-t-il aws_vpc.main.id ? Que se passerait-il si l’on supprimait cette référence ? À quoi sert terraform validate ?

# 6. Étape 3 — Ajouter la sécurité et l’instance EC2

Objectif : compléter le réseau puis créer une instance EC2. L’AMI est recherchée dynamiquement afin d’éviter de coder en dur un identifiant d’image qui varie selon la région.

# 6.1 Créer compute.tf

```
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_security_group" "web" {
  name        = "tp-terraform-web-sg"
  description = "Acces SSH restreint et HTTP"
  vpc_id      = aws_vpc.main.id

  ingress {
    description = "SSH depuis l'IP du stagiaire"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_cidr]
  }

  ingress {
    description = "HTTP"
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "tp-terraform-web-sg"
  }
}

resource "aws_instance" "web" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.public.id
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  tags = {
    Name        = local.instance_name
    Environment = var.environment
  }
}
```

# 6.2 Ajouter les variables et les locals

Complétez variables.tf avec les déclarations suivantes :

```
variable "instance_type" {
  description = "Type de l'instance EC2"
  type        = string
  default     = "t3.micro"
}

variable "environment" {
  description = "Nom de l'environnement"
  type        = string
  default     = "dev"
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "terraform-aws-lab"
}
```

Créez locals.tf :

```
locals {
  instance_name = "${var.project_name}-${var.environment}-web"
}
```

Ici, ${var.project_name} et ${var.environment} sont des interpolations. Terraform construit une chaîne à partir des valeurs des variables. La valeur locale local.instance_name évite de répéter cette expression.

# 6.3 Fournir les valeurs avec terraform.tfvars

```
aws_region         = "eu-west-3"
availability_zone  = "eu-west-3a"
ssh_allowed_cidr   = "VOTRE_IP_PUBLIQUE/32"
instance_type      = "t3.micro"
environment        = "dev"
project_name       = "terraform-aws-lab"
vpc_cidr           = "10.20.0.0/16"
public_subnet_cidr = "10.20.1.0/24"
```

Remplacez VOTRE_IP_PUBLIQUE par votre adresse IPv4 publique réelle suivie de /32. Ne laissez pas cette valeur fictive. Vérifiez aussi que la zone de disponibilité choisie existe dans la région et que votre compte dispose de capacité.

# 6.4 Formater, valider et planifier

```
terraform fmt -recursive
terraform validate
terraform plan
```

Lisez le plan : repérez les ressources qui seront créées, les attributs calculés après création et les éventuelles erreurs. Si le plan est cohérent, demandez l’accord du formateur avant de lancer apply.

```
terraform apply
```

Terraform demande une confirmation. Tapez yes uniquement après avoir vérifié le plan. À la fin, les ressources sont créées dans AWS.

# 6.5 Vérifications dans AWS

```
terraform state list
terraform show
aws ec2 describe-instances --filters "Name=tag:Name,Values=terraform-aws-lab-dev-web" --query "Reservations[].Instances[].[I
```

La commande state list affiche les adresses des ressources suivies dans le State. terraform show présente les détails connus de Terraform.

# 7. Étape 4 — Exposer des informations avec les Outputs

Objectif : afficher des informations utiles à la fin du déploiement.

## 7.1 Créer outputs.tf

```
output "vpc_id" {
  description = "Identifiant du VPC"
  value       = aws_vpc.main.id
}

output "public_subnet_id" {
  description = "Identifiant du sous-reseau public"
  value       = aws_subnet.public.id
}

output "instance_id" {
  description = "Identifiant de l'instance EC2"
  value       = aws_instance.web.id
}

output "instance_public_ip" {
  description = "Adresse IPv4 publique de l'instance"
  value       = aws_instance.web.public_ip
}
```

## 7.2 Appliquer et lire les Outputs

```
terraform apply
terraform output
terraform output instance_public_ip
```

Les Outputs rendent les attributs sélectionnés visibles dans le terminal et accessibles à d'autres configurations ou outils. Une adresse IP publique peut être absente si AWS n'en attribue pas ; ici, le sous-réseau et l'instance demandent une IP publique.

Question : quelle différence entre une valeur calculée par une ressource et un Output ? L'Output crée-t-il une ressource AWS ?

# 8. Étape 5 — Expérimenter les variables

Changez uniquement environment dans terraform.tfvars, par exemple en remplaçant dev par test. Exécutez ensuite :

```
terraform plan
```

Observez les changements proposés. Le nom de l'instance change et certains attributs nécessitent le remplacement de la ressource. Ne lancez apply que si vous comprenez les conséquences.

## 8.1 Surcharger une variable en ligne de commande

```
terraform plan -var="environment=recette"
```

Une valeur passée avec -var surcharge la valeur correspondante du fichier tfvars pour cette exécution.

## 8.2 Comprendre les variables sensibles

L'attribut sensitive masque certaines valeurs dans l'affichage de Terraform, mais ne chiffre pas à lui seul le State. Ne stockez pas de secrets dans un dépôt ou dans un fichier tfvars partagé. Utilisez un gestionnaire de secrets ou des variables d'environnement adaptées.

# 9. Étape 6 — Transformer le projet en module

Objectif : déplacer les ressources dans un module local et appeler ce module depuis le module racine. Un module Terraform est un ensemble de fichiers .tf dans un même répertoire.

## 9.1 Créer l’arborescence

```
tp-terraform-aws/
  versions.tf
  provider.tf
  variables.tf
  terraform.tfvars
  outputs.tf
  main.tf
  modules/
    web-infrastructure/
      main.tf
      variables.tf
      outputs.tf
```

Dans cette étape, nous allons déplacer les déclarations de ressources et la data source dans modules/web-infrastructure/main.tf. Le provider et le bloc terraform restent dans le module racine.

## 9.2 Créer modules/web-infrastructure/variables.tf

```
variable "aws_region" {
  type        = string
  description = "Region AWS"
}

variable "vpc_cidr" {
  type        = string
  description = "CIDR du VPC"
}

variable "public_subnet_cidr" {
  type        = string
  description = "CIDR du sous-reseau public"
}

variable "availability_zone" {
  type        = string
  description = "Zone de disponibilite"
}

variable "ssh_allowed_cidr" {
  type        = string
  description = "CIDR autorise pour SSH"
}

variable "instance_type" {
  type        = string
  description = "Type EC2"
}

variable "environment" {
  type        = string
  description = "Environnement"
}

variable "project_name" {
  type        = string
  description = "Nom du projet"
}
```

## 9.3 Créer modules/web-infrastructure/main.tf

```
locals {
  instance_name = "${var.project_name}-${var.environment}-web"
}

data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]
```

```
  filter {
    name   = "name"
    values = ["al2023-ami-2023.*-x86_64"]
  }

  filter {
    name   = "architecture"
    values = ["x86_64"]
  }

  filter {
    name   = "virtualization-type"
    values = ["hvm"]
  }
}

resource "aws_vpc" "main" {
  cidr_block           = var.vpc_cidr
  enable_dns_support   = true
  enable_dns_hostnames = true

  tags = {
    Name = "${var.project_name}-${var.environment}-vpc"
  }
}

resource "aws_subnet" "public" {
  vpc_id                  = aws_vpc.main.id
  cidr_block              = var.public_subnet_cidr
  availability_zone       = var.availability_zone
  map_public_ip_on_launch = true

  tags = {
    Name = "${var.project_name}-${var.environment}-public-subnet"
  }
}

resource "aws_internet_gateway" "main" {
  vpc_id = aws_vpc.main.id
}

resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id

  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.main.id
  }
}

resource "aws_route_table_association" "public" {
  subnet_id      = aws_subnet.public.id
  route_table_id = aws_route_table.public.id
}

resource "aws_security_group" "web" {
  name   = "${var.project_name}-${var.environment}-web-sg"
  vpc_id = aws_vpc.main.id

  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_allowed_cidr]
  }

  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
```

```
    Name = "${var.project_name}-${var.environment}-web-sg"
  }
}

resource "aws_instance" "web" {
  ami                         = data.aws_ami.amazon_linux.id
  instance_type               = var.instance_type
  subnet_id                   = aws_subnet.public.id
  vpc_security_group_ids      = [aws_security_group.web.id]
  associate_public_ip_address = true

  tags = {
    Name        = local.instance_name
    Environment = var.environment
  }
}
```

# 9.4 Créer modules/web-infrastructure/outputs.tf

```
output "vpc_id" {
  value       = aws_vpc.main.id
  description = "Identifiant du VPC"
}

output "public_subnet_id" {
  value       = aws_subnet.public.id
  description = "Identifiant du sous-reseau public"
}

output "instance_id" {
  value       = aws_instance.web.id
  description = "Identifiant EC2"
}

output "instance_public_ip" {
  value       = aws_instance.web.public_ip
  description = "Adresse IP publique EC2"
}
```

# 9.5 Créer le main.tf du module racine

```
module "web_infrastructure" {
  source = "./modules/web-infrastructure"

  aws_region         = var.aws_region
  vpc_cidr           = var.vpc_cidr
  public_subnet_cidr = var.public_subnet_cidr
  availability_zone  = var.availability_zone
  ssh_allowed_cidr   = var.ssh_allowed_cidr
  instance_type      = var.instance_type
  environment        = var.environment
  project_name       = var.project_name
}
```

Le provider AWS est configuré dans le module racine. Les modules enfants héritent du provider par défaut du parent. La variable aws_region ci-dessus est transmise pour illustrer les entrées du module ; si vous souhaitez l’utiliser dans le module enfant, vous pouvez la supprimer de ses variables car le provider est configuré au niveau racine.

# 9.6 Adapter les Outputs racine

```
output "vpc_id" {
  value = module.web_infrastructure.vpc_id
}

output "instance_id" {
  value = module.web_infrastructure.instance_id
}

output "instance_public_ip" {
  value = module.web_infrastructure.instance_public_ip
}
```

Supprimez les anciens fichiers racine network.tf, compute.tf et locals.tf après avoir déplacé leur contenu dans le module. Les variables d’entrée du module racine restent dans variables.tf.

## 9.7 Initialiser et valider le module

```
terraform init
terraform fmt -recursive
terraform validate
terraform plan
```

Comme les ressources existaient déjà à la racine, leur adresse Terraform change lorsqu’elles sont déplacées dans le module. Sans précaution, Terraform pourrait proposer de détruire les anciennes adresses et de créer les nouvelles. Pour conserver les objets existants, utilisez des blocs moved dans le module racine, ou réalisez cette étape sur un environnement de TP neuf. La méthode moved est détaillée ci-dessous.

# 10. Important — Conserver les ressources lors du déplacement vers un module

Si vous avez déjà appliqué la configuration racine avant la modularisation, les ressources sont enregistrées sous des adresses comme aws_vpc.main. Après déplacement, Terraform les voit sous module.web_infrastructure.aws_vpc.main. Pour indiquer que l'adresse a changé sans que l'objet distant change, ajoutez les blocs moved suivants dans un fichier moved.tf à la racine :

```
moved {
  from = aws_vpc.main
  to   = module.web_infrastructure.aws_vpc.main
}

moved {
  from = aws_subnet.public
  to   = module.web_infrastructure.aws_subnet.public
}

moved {
  from = aws_internet_gateway.main
  to   = module.web_infrastructure.aws_internet_gateway.main
}

moved {
  from = aws_route_table.public
  to   = module.web_infrastructure.aws_route_table.public
}

moved {
  from = aws_route_table_association.public
  to   = module.web_infrastructure.aws_route_table_association.public
}

moved {
  from = aws_security_group.web
  to   = module.web_infrastructure.aws_security_group.web
}

moved {
  from = aws_instance.web
  to   = module.web_infrastructure.aws_instance.web
}
```

Ajoutez également un bloc moved pour toute autre ressource gérée à la racine que vous avez déplacée. Puis exécutez terraform plan. Le plan doit indiquer des déplacements d'adresses et non une destruction/recréation de toutes les ressources. Si le plan propose des remplacements inattendus, arrêtez-vous et analysez-le avant tout apply.

À retenir : le State associe les adresses Terraform aux objets distants. Une refactorisation de code peut donc nécessiter de préserver explicitement cette correspondance.

# 11. Exercices d'approfondissement

• Ajoutez une variable permettant de choisir le port applicatif et utilisez-la dans le Security Group.

• Ajoutez un Output affichant l'identifiant du Security Group.

• Ajoutez une variable booléenne enable_http qui contrôle l'ouverture du port 80. Indice : utiliser un bloc dynamic ou une ressource de règle séparée avec count.

• Créez un second environnement en modifiant le fichier tfvars et observez les différences du plan.

• Appelez le module une seconde fois avec un nom différent, uniquement si vous avez adapté les CIDR et les noms pour éviter les conflits.

# 12. Nettoyage obligatoire

À la fin du TP, supprimez les ressources pour éviter des coûts inattendus. Depuis le répertoire racine, exécutez :

```
terraform plan -destroy
terraform destroy
```

Lisez le plan de destruction, puis confirmez avec yes si vous êtes certain de supprimer uniquement les ressources du TP. Vérifiez ensuite dans AWS que les ressources ont bien disparu. Ne supprimez pas manuellement le fichier terraform.tfstate avant la destruction.

# 13. Dépannage — erreurs fréquentes

| Symptôme | Piste de vérification |
|---|---|
| Unable to locate credentials | Profil AWS, rôle IAM, variable AWS_PROFILE. |
| AccessDenied | Permissions IAM manquantes pour l'API AWS concernée. |
| Invalid availability zone | Zone indisponible dans la région sélectionnée. |
| InvalidParameterValue CIDR | Vérifier le format et le chevauchement des plages CIDR. |
| No AMI found | Vérifier le filtre AMI, l'architecture et la région. |
| SSH inaccessible | IP publique, route vers IGW, Security Group, clé/accès et port 22. |
| Plan veut tout remplacer après modification | Vérifier les blocs moved et les adresses source/cible. |

# Annexe A — Récapitulatif des commandes

```
terraform version          # Version du CLI
terraform init             # Initialiser le projet
terraform fmt -recursive   # Formater les fichiers
terraform validate         # Vérifier la syntaxe et la cohérence
terraform plan             # Prévisualiser les changements
terraform apply            # Appliquer les changements
terraform output           # Afficher les Outputs
terraform state list       # Lister les ressources du State
terraform show             # Afficher l'état courant
terraform plan -destroy    # Prévisualiser la destruction
terraform destroy          # Détruire les ressources gérées
```

# Annexe B — Questions de synthèse

• Expliquez avec vos mots la différence entre impératif et déclaratif.

• Quel est le rôle du provider AWS ?

• Quelle différence faites-vous entre une ressource et une data source ?

• Pourquoi Terraform a-t-il besoin d’un State ?

• Comment une référence entre ressources exprime-t-elle une dépendance ?

• Quelle différence entre variable, local et Output ?

• Qu’apporte un module par rapport à une collection de fichiers à la racine ?

• Pourquoi faut-il examiner le plan avant apply ?

• Pourquoi faut-il protéger le State et ne pas y stocker de secrets inutilement ?

# Annexe C — Points de vigilance formateur

• Le TP crée des ressources réelles dans AWS : confirmer les autorisations, la région, le budget et les quotas avant le démarrage.

• La version exacte de Terraform du poste n’a pas été fournie. Le fichier versions.tf accepte Terraform >= 1.5.0 et < 2.0.0 ; il ne prouve pas que chaque version future est compatible. Tester le TP avec la version réellement installée.

• Le provider AWS est contraint à la branche majeure 6.x. terraform init sélectionnera une version admissible et .terraform.lock.hcl figera la sélection pour le projet.

• La configuration de sécurité est un exemple pédagogique, pas une architecture de production. Pour un vrai service, renforcer IAM, SSH, journalisation, chiffrement, segmentation réseau et gestion des secrets.

• Le TP ne crée pas de clé EC2. La connexion SSH nécessite une méthode d’accès configurée séparément, par exemple une clé existante ou AWS Systems Manager. La partie SSH est facultative.

• Le réseau est public et l’instance reçoit une IPv4 publique : cela peut générer des frais selon les conditions tarifaires AWS en vigueur.

Fin du TP — Les stagiaires ont construit une infrastructure AWS, utilisé variables et Outputs, puis structuré le code dans un module local.